Aarya Arban

T11 – 05

Assignment No. 6

Aim: Creating Decision Tree using Python

Program:

```
import pandas as pd
import numpy as np
from math import log2

df = pd.read_csv('Buy_Computer.csv')

def entropy(df, target_column):
    value_counts = df[target_column].value_counts()
    total_records = len(df)
    entropy_value = 0
    for count in value_counts:
        prob = count / total_records
        entropy_value -= prob * log2(prob)
    return entropy_value

def information_gain(df, attribute, target_column):
    total_entropy = entropy(df, target_column)
    attribute_values = df[attribute].value_counts()
    weighted_entropy = 0

    for value, count in attribute_values.items():
```

```
        subset = df[df[attribute] == value]
        weighted_entropy += (count / len(df)) * entropy(subset,
    target_column)

    return total_entropy - weighted_entropy

# First iteration
target_column = 'Buy_Computer'
attributes = ['age', 'income', 'student', 'credit_rating']

# Calculate Information Gain for each attribute
information_gains = {}
for attribute in attributes:
    information_gains[attribute] = information_gain(df, attribute,
     target_column)

# Find the best attribute to split on (first iteration split by 'age')
best_attribute = max(information_gains, key=information_gains.get)

# Split based on the best attribute 'age' and print subsets
subsets = {}
for value in df[best_attribute].unique():
    subset = df[df[best_attribute] == value]
    subsets[value] = subset

# Print Information Gain for first iteration (age)
print("Information Gain for each attribute (First Iteration):")
```

```
for attribute, gain in information_gains.items():
    print(f"{attribute}: {gain}")
print(f"\nBest attribute to split on: {best_attribute}")
print("\nThe dataset is split based on the attribute:", best_attribute)

# Now, we will process only 'youth' and 'senior' subsets for the second
    iteration
print("\nSecond Iteration for youth and senior subsets:")
for value in ['youth', 'senior']:  # Exclude 'middle_age' subset
    subset = subsets[value]
    print(f"\nSecond Iteration for subset where {best_attribute} = {value}:")

    # Exclude 'age' from the remaining attributes (since 'age' was already
     used)
    remaining_attributes = [attr for attr in attributes if attr != best_attribute]

    # Calculate Information Gain for each of the remaining attributes in this
     subset
    information_gains_2 = {}
    for attribute in remaining_attributes:
        information_gains_2[attribute] = information_gain(subset, attribute,
     target_column)

    # Find the best attribute to split on within the current subset
    best_attribute_2 = max(information_gains_2,
     key=information_gains_2.get)
    print(f"\nInformation Gain for each attribute in {value} subset:")
    for attribute, gain in information_gains_2.items():
```

```
        print(f"{attribute}: {gain}")

    print(f"\nBest attribute to split on in this subset: {best_attribute_2}")

    # Print the rule for splitting
    print(f"\nRule for splitting {value} subset: Split by {best_attribute_2}")

    # Split the subset based on the best attribute and print the results
    print(f"\nThe dataset is split based on the attribute: {best_attribute_2}")
    for val in subset[best_attribute_2].unique():
        subset_split = subset[subset[best_attribute_2] == val]
        print(f"\nSubset where {best_attribute_2} = {val}:")
        print(subset_split)

        # Display the rule for this subset
        print(f"\nRule for {best_attribute_2} = {val}: If {best_attribute_2} = {val},
      then {target_column} = {'yes' if subset_split[target_column].iloc[0] ==
      'yes' else 'no'}")

# Automatically recognize if a subset has a pure class
def check_pure_subset(df, target_column):
    return len(df[target_column].unique()) == 1

# Store tree structure
decision_tree = {}

def build_tree(subset, parent_attribute="Root"):
```

```
    if check_pure_subset(subset, target_column):
        decision_tree[parent_attribute] = subset[target_column].iloc[0]
        return

    remaining_attributes = [attr for attr in attributes if attr not in
     decision_tree]
    if not remaining_attributes:
        return

    information_gains_local = {attr: information_gain(subset, attr,
     target_column) for attr in remaining_attributes}
    best_attr = max(information_gains_local,
     key=information_gains_local.get)
    decision_tree[parent_attribute] = best_attr

    for val in subset[best_attr].unique():
        subset_split = subset[subset[best_attr] == val]
        build_tree(subset_split, f"{best_attr} = {val}")

# Build the tree
for value, subset in subsets.items():
    build_tree(subset, f"{best_attribute} = {value}")

# Print the decision tree
print("\nDecision Tree Representation:")
for key, value in decision_tree.items():
    print(f"{key} -> {value}")
```

Output:

Information Gain for each attribute (First Iteration):

age: 0.24674981977443933

income: 0.02922256565895487

student: 0.15183550136234159

credit_rating: 0.04812703040826949

Best attribute to split on: age

The dataset is split based on the attribute: age

Second Iteration for youth and senior subsets:

Second Iteration for subset where age = youth:

Information Gain for each attribute in youth subset:

income: 0.5709505944546686

student: 0.9709505944546686

credit_rating: 0.01997309402197489

Best attribute to split on in this subset: student

Rule for splitting youth subset: Split by student

The dataset is split based on the attribute: student

Subset where student = no:

	id	age	income	student	credit_rating	Buy_Computer
0	1	youth	high	no	fair	no
1	2	youth	high	no	excellent	no
7	8	youth	medium	no	fair	no

Rule for student = no: If student = no, then Buy_Computer = no

Subset where student = yes:

	id	age	income	student	credit_rating	Buy_Computer
8	9	youth	low	yes	fair	yes
10	11	youth	medium	yes	excellent	yes

Rule for student = yes: If student = yes, then Buy_Computer = yes

Second Iteration for subset where age = senior:

Information Gain for each attribute in senior subset:

income: 0.01997309402197489

student: 0.01997309402197489

credit_rating: 0.9709505944546686

Best attribute to split on in this subset: credit_rating

Rule for splitting senior subset: Split by credit_rating

The dataset is split based on the attribute: credit_rating

Subset where credit_rating = fair:

id	age	income	student	credit_rating	Buy_Computer
3	4	senior	medium	no	fair	yes
4	5	senior	low	yes	fair	yes
9	10	senior	medium	yes	fair	yes

Rule for credit_rating = fair: If credit_rating = fair, then Buy_Computer = yes

Subset where credit_rating = excellent:

id	age	income	student	credit_rating	Buy_Computer
5	6	senior	low	yes	excellent	no
13	14	senior	medium	no	excellent	no

Rule for credit_rating = excellent: If credit_rating = excellent, then Buy_Computer = no

Decision Tree Representation:

age = youth -> student

student = no -> no

student = yes -> yes

age = middle -> yes

age = senior -> credit_rating

credit_rating = fair -> yes

credit_rating = excellent -> no